MIT 8103 Advanced Database Systems

MIT 8103 laboratory equipment booking exercise · Kehinde Irhodia · Student ID 301836304

Case study

My case study is a fictional university laboratory equipment booking system. It stores members, equipment categories, individually bookable units, booking requests and returns. Laboratory members request equipment, while technicians record approvals, returns and condition notes. The five related tables preserve booking history and support queries about equipment use. Future reservation scheduling and application workflow are outside this prototype.

Portfolio 1: Database Design and Modelling

I organised the relational design into 5 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. A partial unique index allows only one approved or checked-out request per unit. Pending and historical requests do not participate in that index. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 16 deliberately invalid inserts were rejected without changing the valid sample rows.

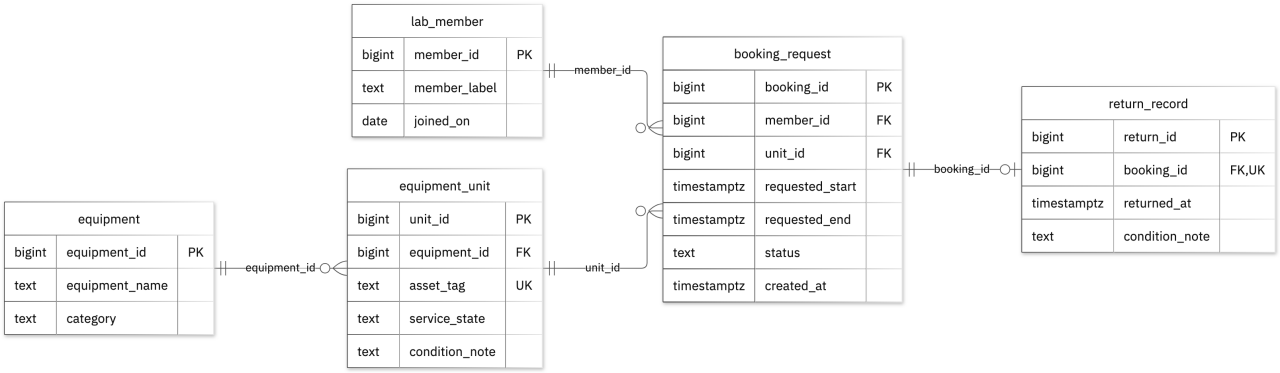


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries against the sample records. The join returned one row and included Fictional Member Alpha with SYN-001. For the index comparison, I used 20,000 separate performance rows. The median lookup time changed from 0.928 ms to 0.111 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on booking_request to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used lab_booking_events for the case's event documents. MongoDB inserted 2 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/elcharitas/Irhodia_Kehinde_301836304/tree/main

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots. The link identifies the submission revision on **main**.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for lab_booking_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I kept lab_member, equipment, equipment_unit, booking_request and return_record in separate tables so their details could be maintained without repeating them in every related record. A partial unique index allows only one approved or checked-out request per unit. Pending and historical requests do not participate in that index. The constraint tests check this alongside the required values and record links. One issue illustrated by the two-session exercise is that both sessions can read the same value before either update is committed. Locking the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

OpenAI Codex generated the case model and execution data through the local portfolio generator. PostgreSQL and MongoDB produced the saved results. Evidence images render those recorded results